Laboratory Exercise 3

Memory-Mapped I/O, Polling and Timers

The purpose of this exercise is to explore the use of devices that provide input and output capabilities for a processor. We assume that you are using the Nios V processor in the DE1-SoC Computer. A good approach is to first implement each part of this exercise by using the *CPUlator* simulator, and then to implement your solution in a hardware board, if available. If a hardware system other than the DE1-SoC Computer is being used, then some parts of this exercise may need to be modified to suit the features of your board.

There are two basic techniques for synchronizing with I/O devices: program-controlled *polling* and *interrupt-driven* approaches. We will use the polling approach in this exercise, writing programs in the Nios V assembly language. *Parallel port* interfaces, as well as a *timer* hardware module, will be used as examples of I/O hardware.

In general, a *parallel port* provides for data transfer in either the input or output direction. The transfer of data to/from the processor is done in parallel and may involve from 1 to 32 bits. The number of bits, n, and the type of transfer depend on the specific parallel port being used. The parallel port interface can contain the four registers shown in Figure 1.

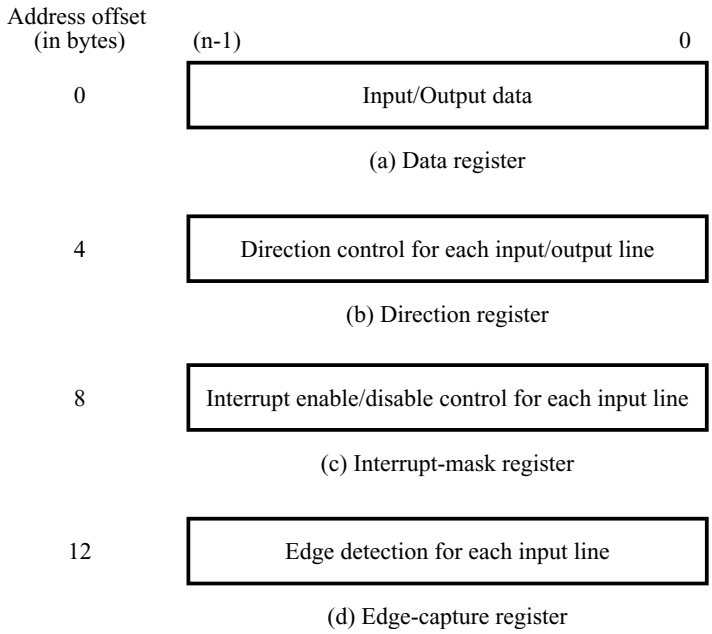

Figure 1: Registers in the parallel port interface.

Each register is n bits long. The registers have the following purpose:

- *Data* register: holds the n bits of data that are transferred between the parallel port and the Nios V processor. The *Data* register can be implemented as an input, output, or a bidirectional port.
- *Direction* register: defines the direction of transfer for each of the n data bits when a bidirectional interface is generated.
- *Interrupt-mask* register: used to enable interrupts from the input lines connected to the parallel port. This register will be described in more detail in a later laboratory exercise.

- *Edge-capture* register: indicates when a change of logic value is detected in the signals on the input lines connected to the parallel port. Once a bit in the edge capture register becomes asserted, it will remain asserted. An edge-capture bit can be de-asserted by writing to it using the Nios V processor.

Not all of the registers depicted in Figure 1 are present in some parallel ports. For example, the *Direction* register is included only when a bidirectional interface is present in the parallel port. The *Interrupt-mask* and *Edge-capture* registers must be included if interrupt-driven input/output (described in a later laboratory exercise) is used.

The parallel port registers are memory mapped, starting at a specific *base* address. The base address becomes the address of the *Data* register in the parallel port. The addresses of the other three registers have offsets of 4, 8, or 12 bytes (1, 2, or 3 words) from this base address. In the DE1-SoC Computer parallel ports are used to connect to several types of hardware resources, including *SW* slide switches, *KEY* pushbuttons, *LEDs*, and seven-segment displays.

Conforming to the Nios V Procedure Call Standard

The Nios V *Procedure Call Standard* (PCS) provides guidelines ("rules") for how registers should be used in a program, especially when subroutines are involved. The PCS states that parameters should be passed to a subroutine by using the Nios V *argument* registers *a0* to *a7*. Results should be returned from a subroutine in registers *a0* and *a1*. The PCS states that a subroutine can freely modify the Nios V *temporary* registers *t0* to *t6*, but it is not permitted to have changed the contents of the Nios V *save* registers *s0* to *s11* after execution of the subroutine. If these save registers need to be changed, then the register values should first be saved to memory and later restored before returning from the subroutine. (Note: if you are using *CPUlator*, then you will see an error stating that you have "clobbered" one ore more of registers *s0* to *s11* if you over-write these registers in a subroutine without saving/restoring their values as required by the PCS.)

You should always follow the Nios V PCS in all assembly-language code that you write.

Part I

The DE1-SoC Computer contains several parallel ports that are connected to simple switches, lights, and displays. Figure 2 depicts the parallel port connected to the ten red lights, called *LEDR*, on the DE1-SoC board. Software code can turn the lights on or off by writing the value 1 or 0, respectively, to each of the ten bit-positions in the register that is mapped to address `0xFF200000`.

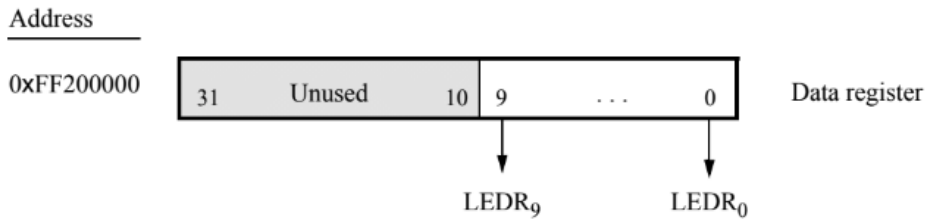

Figure 2: The parallel port connected to the red lights *LEDR*.

The parallel port connected to the ten slide switches, called *SW*, on the DE1-SoC board is shown in Figure 3. Software code can load the state of the switches by reading from the register mapped to address `0XFF200040`.

The DE1-SoC Computer has two parallel ports connected to the seven-segment displays on the DE1-SoC board. The port connected to the displays *HEX* 3-0 is memory mapped at the base address `0xFF200020` and the other parallel port is connected to *HEX* 5-4, at the base address `0xFF200030`. Figure 4 shows how the display segments are connected to the parallel ports.

An example of code that uses the *LEDR*, *SW*, and *HEX* 3-0 parallel ports is given in Figure 5. This code reads

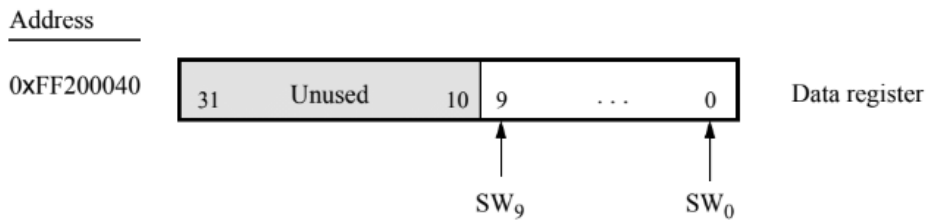

Figure 3: The parallel port connected to the *SW* switches.

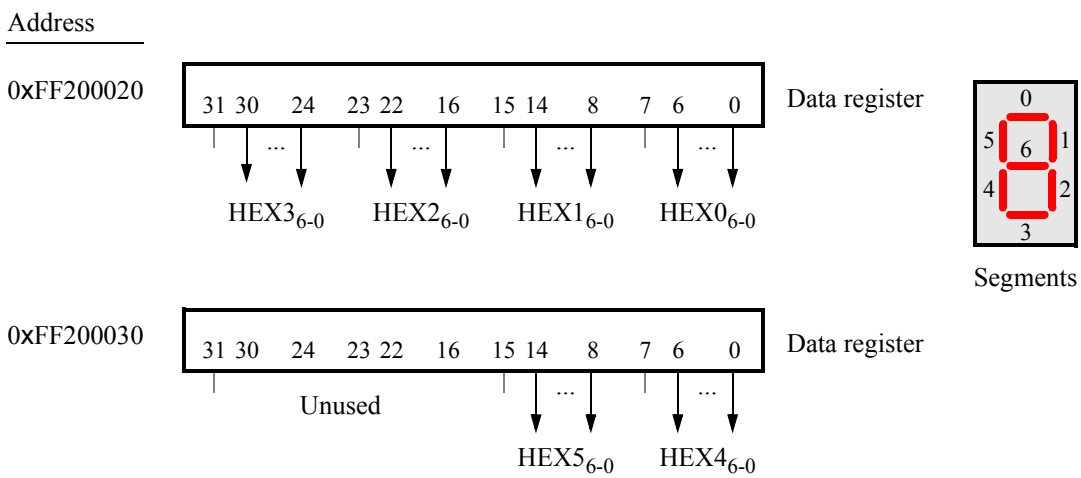

Figure 4: The parallel ports connected to the seven-segment displays *HEX* 5-0.

from the *SW* switches and writes the resulting value to both the *LEDR* and *HEX* 3-0 ports. Compile and execute this code to experiment with different settings of the *SW* switches, and observe the *LEDR* and *HEX* 3-0 displays.

```
# Program that displays SW switch settings on LEDR and HEX0

.equ LEDR_BASE, 0xff200000
.equ HEX3_HEX0_BASE, 0xff200020
.equ SW_BASE, 0xff200040

.global _start
_start: li      t0, LEDR_BASE       # pointer to LEDR port
        li      t1, SW_BASE         # pointer to SW port
        li      t2, HEX3_HEX0_BASE  # pointer to HEX port

loop:   lw      t3, (t1)            # read from SW
        sw      t3, (t0)            # write to LEDR
        sw      t3, (t2)            # write to HEX0
        j       loop
```

Figure 5: Assembly-language program that uses the parallel ports.

Part II

You are to write a Nios V program that displays a hexadecimal digit (0, 1, . . ., 9, A, b, C, d, E, F) on the seven-segment display *HEX*0 as described below. The other seven-segment displays $HEX5 - 1$ should be set to be blank.

The following functionality should be included:

- If KEY_0 is pressed on the board, you should set the number displayed on *HEX*0 to 0.
- If KEY_1 is pressed then you should increment the displayed number, but don't let the number go above f (i.e. pressing the key won't change the value if it is already at f).
- If KEY_2 is pressed then decrement the number, but don't let the number go below 0 (i.e. pressing the key won't change the value if it is already 0).
- Pressing KEY_3 should blank the display, and pressing any other KEY after that should return the display to 0.
- The parallel port connected to the pushbutton *KEYs* has the base address 0xFF200050, as illustrated in Figure 6. In your program, use the *polling* I/O method to read the *Data* register to see when a button is being pressed.
- When you are not pressing any *KEY* the *Data* register provides 0, and when you press KEY_i the *Data* register provides the value 1 in bit position i. Once a button-press is detected, be sure that your program waits until the button is released. You *must not* use the *Interruptmask* or *Edgecapture* registers for this part of the exercise.

Implement your code in a file called *part2.s*. Compile and execute your code and test it thoroughly according to the above specification.

Address	31 30 . . . 4	3 2 1 0	
0xFF200050	Unused	$KEY_{3\text{-}0}$	Data register
Unused	Unused		
0xFF200058	Unused	Mask bits	Interruptmask register
0xFF20005C	Unused	Edge bits	Edgecapture register

Figure 6: The parallel port connected to the pushbutton *KEYs*.

Part III

Write a Nios V assembly language program that displays a two-digit hexadecimal *counter* on the seven-segment displays *HEX* 1-0. The counter should be incremented approximately every one second. When the counter reaches the value FF, it should start again at 00.

To achieve a delay of approximately one second, use a delay-loop in your assembly language code. A suitable example of such a loop is shown below, which gives a good value for the delay when using the simulated Nios V processor in the *CPUlator* tool. If you are using the real Nios V processor on a DE1-SoC board, then a much smaller delay value has to be used, because the Nios V processor executes code *much* more quickly in hardware than in the *CPUlator* simulation.

```
delay:      li      t0, 15000000        # wait for a little while
dloop:      addi    t0, t0, -1
            bnez    t0, dloop
```

To avoid "missing" any button presses while the processor is executing the delay loop, you should use the *Edgecapture* register in the *KEY* port, shown in Figure 6. When a pushbutton is pressed, the corresponding bit in the *Edgecapture* register is set to 1; it remains set until your program resets it back to 0. You reset an *Edgecapture* bit by writing a 1 into the corresponding position of the register.

Put your code into a file called *part3.s*, and test and debug your program.

Part IV

In Part III you used a delay loop to cause the Nios V processor to wait for approximately one second. The processor loaded a large value into a register before the loop, and then decremented that value until it reached 0. In this part you are to modify your code so that a *hardware timer* is used to measure an exact delay of one second.

The DE1-SoC Computer includes a number of hardware timers, but for this exercise you will use only the *Machine Timer*, which is part of the Nios V processor. As illustrated in Figure 7 the Machine Timer comprises two 64-bit registers. The *mtime* register is memory-mapped to address `0xFF202100`, and the *mtimecmp* register is mapped to address `0xFF202108`. Since these registers are 64 bits in width, two Nios V memory accesses are needed to access either register. The Nios V processor can interact with the Machine Timer two mains ways: using interrupts, or using polled-I/O. In this exercise you will use the polled-I/O method.

When using polled-IO, an easy way to interact with the Machine Timer is as follows. First, you store a *timeout* value into the *mtimecmp* register. If this timeout value fits within a 32-bit unsigned number, then you can just store it into the low word of *mtimecmp*, at address `0xFF202108`, and set the high-word of *mtimecmp*, at address `0xFF20210C`, to 0. The Machine Timer is always running, and its current value is reflected in the *mtime* register. The Machine Timer increments at the rate of 100,000,000 cycles per second in the DE1-SoC Computer. Each time the value of the *mtime* register reaches the value stored in the *mtimecmp* register, then a *timeout* has occurred. Once this happens, you can begin the next timeout period by resetting the value of the *mtime* register to 0.

Store your code for this part of the exercise in a file called *part4.s*. It should be the same as in Part III, except that you should initialize the *mtimecmp* and *mtime* registers at the beginning of your code. Set the *mtimecmp* register so as to generate a timeout every one second. In your program's main loop, poll on the value of the *mtime* register to wait for each timeout period, rather than decrementing a register as in Part III. Test and debug your program.

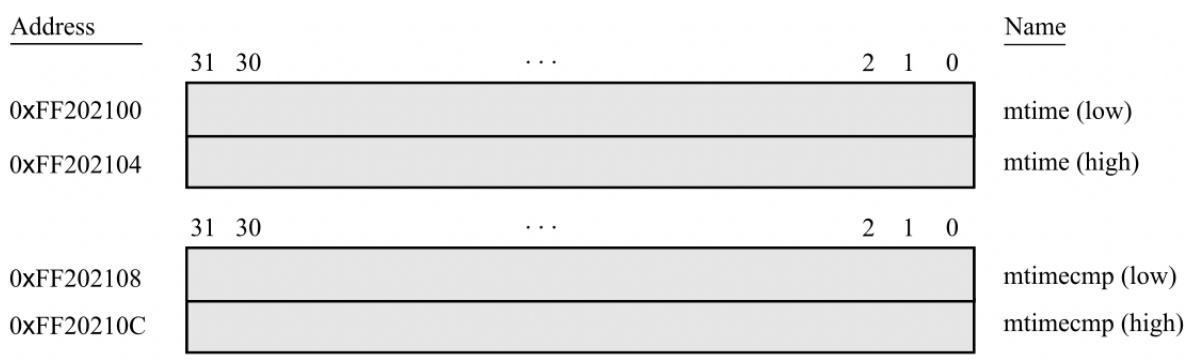

Figure 7: The Nios V Machine Timer registers.

Copyright © FPGAcademy.org. All rights reserved. FPGAcademy and the FPGAcademy logo are trademarks of FPGAcademy.org. This document is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement. In no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the document or the use or other dealings in the document.

*Other names and brands may be claimed as the property of others.